

软件概要设计说明书

软件概要设计说明书

1 引言

1.1 编写目的

1.2 背景

1.3 定义

1.4 参考设计

2 系统需求概述

2.1 需求规定

2.2 业务目标

2.3 运行环境

2.3.1 设备

2.3.2 支持软件

2.4 设计约束

2.5 功能需求

2.6 非功能需求

2.7 系统总体架构

3 系统接口设计

3.2 接口设计

3.2.1 用户接口

3.2.2 与其他软件、硬件接口

3.3 系统数据结构设计

3.3.1 逻辑结构设计要点

3.3.2 物理结构设计要点

3.3.3 数据结构与程序关系

4 系统数据库设计

5 运行设计

5.1 运行模块组合

5.2 运行控制

5.3 运行时间

1 引言

1.1 编写目的

本文档为在线视频与直播网站的概要设计说明书，主要目的为概要说明对程序系统整体与部分的设计考虑，包括程序系统的基本处理流程、程序系统的组织结构、初步的模块划分、功能分配、接口设计、运行设计、安全设计、数据结构设计和异常处理设计等，同时为程序后续的详细设计提供基础。

本文档预期读者为开发小组成员及课程教师与助教。开发人员可依据本文档进行详细设计和编码；课程教师和助教可依据本文档审查系统设计是否覆盖需求。

1.2 背景

- 待开发软件系统名称：在线视频与直播网站。
- 项目任务提出者：软件工程基础课程课程组。
- 开发者：软件工程基础课程9组。
- 用户：内容创作者、观看用户、平台运营管理人员。
- 运行环境：Web浏览器、服务器。

1.3 定义

API: Application Programming Interface, 应用程序接口。

CDN: Content Delivery Network, 内容分发网络。

CSCI: Computer Software Configuration Item, 计算机软件配置项。

DAO: Data Access Object, 数据访问对象。

DTO: Data Transfer Object, 数据传输对象。

FFmpeg: 开源音视频处理工具, 用于视频转码、截图、压缩等。

HLS: HTTP Live Streaming, 一种常见 HTTP 流媒体协议。

JWT: JSON Web Token, 一种常见身份令牌格式。

RTMP: Real-Time Messaging Protocol, 实时消息传输协议, 常用于直播推流。

SRS: Simple Realtime Server, 开源流媒体服务器。

RESTful API: 一种接口设计风格, 使用HTTP方法表示操作类型。

WebSocket: 基于 TCP 的全双工通信协议, 可用于实时弹幕、直播聊天等场景。

1.4 参考设计

本文档参考了以下文档:

- 《软件工程基础2026春大作业要求》
- 《9组-软件开发计划书》
- 《9组-软件需求规格说明书》
- 《软件概要设计说明书》文档模版

2 系统需求概述

2.1 需求规定

本系统主要输入输出项目与处理功能如下:

输入项目:

- 用户注册信息, 包括手机号或邮箱、密码、验证码。
- 用户登录凭证, 包括账号、密码。
- 视频文件, 包括MP4、AVI、MOV等可能格式。
- 视频元信息, 包括标题、描述、标签、封面图片。
- 直播配置信息, 包括直播间标题、分类、封面。
- 弹幕内容, 包括文本、颜色、位置。
- 评论内容, 包括文本、目标视频、父评论。
- 搜索关键词, 包括文本、筛选条件。
- 关注操作, 包括目标用户ID。

输出项目:

- 视频播放流, 包括清晰度自适应流。
- 推荐视频列表, 包括封面、标题、播放量、作者信息。
- 搜索结果列表, 包括匹配视频及相关性排序。
- 用户个人信息, 包括头像、昵称、粉丝数、获赞数。

- 直播间画面，包括实时视频流、在线人数、弹幕。
- 弹幕消息，包括实时推送的文本弹幕。
- 创作者统计数据，包括播放量趋势、粉丝增长、收益。

处理功能：

- 用户注册登录，包括身份验证、密码加密、会话管理。
- 视频上传转码，包括分片接收、格式转换、多清晰度生成、封面提取。
- 视频播放，包括地址解析、清晰度选择、断点续播、历史记录。
- 直播推流观看，包括推流地址分配、实时分发、连麦支持。
- 弹幕实时显示，包括消息接收、内容过滤、广播分发。
- 内容推荐，包括基于内容的标签匹配、热度计算、新内容发现。
- 搜索过滤，包括全文检索、多维度排序、结果分页。
- 关注互动，包括关系建立、动态推送、粉丝管理。
- 创作者数据分析，包括指标统计、趋势图表、内容管理。

2.2业务目标

基于课程大作业要求，本系统的主要业务目标为：连接内容创作者与观看者，打造集短视频、中长视频、直播互动于一体的综合视频平台。注重推荐精准性、分发公平性和社区健康度，让每一种表达都被看见，让每一次观看都有收获。

2.3运行环境

2.3.1设备

开发设备：个人PC，4核或以上CPU、8GB或以上内存、256GB或以上硬盘，用于编码开发和本地测试。

测试服务器：单台云服务器或本地服务器，2核或以上CPU、4GB或以上内存、100GB或以上硬盘、5Mbps或以上带宽，用于部署测试环境及可能的演示。

生产服务器（预期）：2核或以上CPU、8GB或以上内存、200GB或以上硬盘、10Mbps或以上带宽，用于支持更多并发用户。

用户设备：通过Chrome、Firefox、Edge等主流浏览器访问。

2.3.2支持软件

操作系统：开发机可使用Windows 10+。

数据库：MySQL 8.0，用于持久化存储业务数据。

缓存：Redis 6.0，用于会话管理和热点数据缓存。

开发框架：Spring Boot 2.7+（后端）、Vue.js 3.0+（前端）。

构建工具：Maven 3.6+（Java项目构建）、npm（前端依赖管理）。

版本控制：Git，代码托管于GitHub或GitLab。

2.4设计约束

项目周期：3个月（2026年4月至2026年6月）。

时间约束：严格遵照课程给定各时间节点。

团队规模：5人，功能范围需控制在可实现、可演示、可测试的范围内。

预算约束：课程大作业，无商业预算，优先采用开源组件和本地服务器部署。

2.5功能需求

本系统采用单体架构，前后端分离设计。用户通过浏览器访问前端页面，前端调用后端RESTful接口获取数据和执行业务操作。后端处理业务逻辑，读写数据库和缓存，返回统一格式的JSON结果。

各基本处理流程详细说明如下：

视频浏览流程：用户打开网站首页，前端发送请求获取推荐视频列表。后端接收请求，判断用户是否登录：已登录用户查询其兴趣标签，从数据库匹配相关视频并按算法排序；未登录用户返回全站热门视频。后端将视频元数据（ID、标题、封面URL、播放量、作者信息）封装为JSON返回。前端解析数据，渲染视频卡片网格布局。用户滚动页面至底部时，前端自动加载下一页数据。

视频播放流程：用户点击视频封面，前端路由跳转至播放页，携带视频ID参数。页面加载时，前端并行发送多个请求：获取视频详细信息（标题、标签、作者、相关视频）、获取视频播放地址（多清晰度URL列表）、获取历史弹幕数据、获取评论列表。后端分别处理：查询视频表获取元信息；根据视频ID拼接文件存储路径返回URL；查询弹幕表前100条记录；查询评论表按时间排序。前端播放器根据网络状况默认选择合适清晰度，用户可手动切换。播放过程中，前端定时上报播放进度，用于记录历史。

视频上传流程：创作者进入上传页，选择本地视频文件。前端可使用分片上传技术，将大文件切分为多个小块依次上传，失败时可重传单个分片而非整个文件。后端可接收分片，暂存于临时目录，全部接收后合并为完整文件。后端调用FFmpeg进行转码处理：提取视频信息（时长、分辨率、码率）；生成封面截图（视频第5秒画面）；转码为480P、720P、1080P三种清晰度版本；将各版本移至正式存储目录。后端将视频元信息写入数据库，状态设为待审核。审核通过后（人工或自动），视频加入推荐池。

直播开启流程：创作者点击开播按钮，前端弹出配置窗口填写直播间标题、选择分类、上传封面。确认后，前端发送创建请求。后端生成唯一直播间ID，调用直播服务器管理API创建直播间，获取推流地址（RTMP协议URL）。后端将直播间信息写入数据库，状态设为准备中。前端显示推流地址和二维码，创作者使用OBS等推流工具或浏览器原生推流开始推送视频流。推流开始后，直播服务器通知后端，后端更新直播间状态为直播中，开始向观众提供服务。

直播观看流程：观众通过链接或搜索进入直播间页面。前端加载时，发送请求获取直播间信息（标题、主播、在线人数）。后端返回信息，同时建立WebSocket长连接用于实时通信。前端使用FLV.js或HLS.js播放器拉取直播流地址，开始播放。观众发送弹幕时，前端通过WebSocket发送消息，后端接收后存入数据库，并广播至同直播间所有连接的客户端。观众进入、离开、点赞时，后端通过WebSocket推送通知，更新在线人数显示。

2.6非功能需求

性能需求：页面首次加载时间不超过3秒（4G网络环境下）；视频首帧加载时间不超过2秒；主要API接口响应时间不超过500毫秒；直播延迟（推流到观看）不超过5秒。

并发需求：支持至少100人同时在线观看同一直播；普通视频播放并发受限于服务器带宽，目标支持50人同时播放不同视频。

可用性需求：视频播放等系统核心功能可用性不低于99%。

扩展性需求：数据库可设计预留字段，接口可设计预留扩展参数，便于后续功能增加。

安全性需求：用户密码应加密存储；敏感操作需身份验证；防止SQL注入和XSS攻击；视频文件访问需授权校验。

2.7系统总体架构

本系统总体架构采用三层架构模式，分为表现层、业务逻辑层和数据访问层。

表现层：

表现层是用户直接交互的界面，负责页面渲染、用户输入收集、交互反馈和向后端发送请求。

前端采用Vue.js 3.0框架开发单页应用（SPA），通过Vue Router实现页面路由切换，通过Vuex管理全局状态（用户信息、播放状态）。

主要页面可包括：首页（推荐视频流）、发现页（分类浏览和搜索）、播放页（视频播放和互动）、直播页（直播观看和弹幕）、个人中心（用户信息和管理）、创作者中心（数据分析和内容管理）、上传页（视频上传和编辑）、开播页（直播配置和推流）等。

UI组件库选用Element Plus或Ant Design Vue，提供一致的视觉风格和交互体验。

业务逻辑层：

业务逻辑层处理核心业务规则，是系统的核心处理引擎，采用Spring Boot 2.7框架开发，按功能可划分为以下模块：

- 用户模块：处理注册、登录、信息查询、权限校验。
- 视频模块：处理上传接收、转码调度、播放地址生成、搜索查询。
- 直播模块：处理直播间管理、推流地址分配、状态监控、结束清理。
- 弹幕模块：处理消息接收、内容过滤、广播分发、历史查询。
- 推荐模块：处理兴趣分析、内容匹配、热度计算、结果排序。
- 互动模块：处理评论、点赞、关注、私信。
- 创作者模块：处理数据统计、内容管理、粉丝互动工具。

模块间通过内部方法调用协作，共享数据库连接和缓存客户端。

数据访问层：

数据访问层负责数据的持久化存储、缓存管理和文件存取。

- MySQL数据库：存储用户、视频、直播、弹幕、评论、关注等业务数据。采用连接池管理数据库连接，提高访问效率。
- Redis缓存：存储用户会话令牌、视频热度排行、热门视频列表、频繁访问的用户信息。设置合理的过期时间，平衡命中率和内存占用。
- 文件存储：视频文件和封面图片存储于服务器本地文件系统或云存储服务。按规则组织目录结构，便于管理和迁移。

3 系统接口设计

3.2接口设计

3.2.1用户接口

用户接口是用户通过浏览器与系统交互的界面，包括页面布局、操作命令和系统反馈信息。

- 页面布局命令：

用户访问首页时，浏览器发送 GET 请求至根路径 `/`，服务器返回首页 HTML 页面，页面包含顶部导航栏、分类标签栏和主体视频流区域。导航栏左侧为网站 Logo（点击返回首页），中间为搜索输入框（占位符文本为“搜索视频、直播、用户”），右侧为用户入口（未登录显示“登录”和“注册”按钮，已登录显示用户头像下拉菜单）。

用户点击视频卡片时，浏览器路由跳转至 `/video/{videoId}`，页面加载视频播放页，包含播放器区域、视频信息区和互动区。播放器宽度自适应容器，高度按 16:9 比例，控制栏包含播放/暂停按钮、进度条、当前时间/总时长显示、清晰度切换按钮（显示当前清晰度如"720P"）、倍速选择（0.5x/1x/1.5x/2x）、音量滑块、全屏按钮。

用户进入直播间时，浏览器路由跳转至 `/live/{roomId}`，页面加载直播间，上部为直播播放器，右侧或下部为弹幕显示区和输入框，输入框带颜色选择器和发送按钮。

- 操作命令语法结构：

注册命令：用户点击"注册"按钮，浏览器弹出模态框，包含手机号/邮箱输入框（`type="text"`，必填，占位符"请输入手机号或邮箱"）、密码输入框（`type="password"`，必填，占位符"请输入6-20位密码"）、验证码输入框（`type="text"`，必填，占位符"请输入验证码"）和"获取验证码"按钮、提交按钮（`type="submit"`，文本"注册"）。用户填写后点击提交，浏览器发送 POST 请求至 `/api/auth/register`，请求体为 JSON 格式：`{"account": "13800138000", "password": "*****", "verifyCode": "123456"}`。

登录命令：用户点击"登录"按钮，浏览器弹出模态框，包含账号输入框（`type="text"`，占位符"手机号/邮箱"）、密码输入框（`type="password"`，占位符"密码"）、提交按钮（文本"登录"）。用户填写后点击提交，浏览器发送 POST 请求至 `/api/auth/login`，请求体为 JSON 格式：`{"account": "13800138000", "password": "*****"}`。

搜索命令：用户在顶部搜索框输入关键词，按回车键或点击搜索图标，浏览器发送 GET 请求至 `/api/search/videos?keyword=关键词&page=1&pageSize=20`，服务器返回搜索结果列表，浏览器跳转至搜索页 `/search?keyword=关键词`，渲染结果。

上传命令：创作者点击"上传"按钮，浏览器跳转至 `/upload`，页面显示文件选择区域（支持点击选择和拖拽上传，提示文本"拖拽视频至此或点击上传"）、视频信息填写表单（标题输入框、描述文本域、标签输入框支持自动补全、封面选择区域支持自动截取或本地上传）、提交按钮（文本"发布"）。用户选择文件后，前端开始分片上传，每片发送 POST 请求至 `/api/videos/upload`，请求体为 `multipart/form-data` 格式，包含分片序号、总分片数和文件块数据。全部分片上传完成后，前端发送 POST 请求至 `/api/videos`，请求体为 JSON 格式，包含视频元信息。

开播命令：创作者点击"开播"按钮，浏览器跳转至 `/live/start`，页面显示直播间配置表单（标题输入框、分类下拉选择、封面上传区域）、确认开播按钮。用户填写后点击确认，浏览器发送 POST 请求至 `/api/live/rooms`，请求体为 JSON 格式：`{"title": "直播间标题", "category": "游戏", "cover": "封面URL"}`。服务器返回直播间信息和推流地址，页面显示推流地址（可复制）和二维码。

弹幕命令：用户在播放页或直播页底部输入框输入文本，选择颜色（默认白色，可选红色、黄色、蓝色、绿色），选择位置（默认滚动，可选顶部、底部），点击发送按钮或按回车键。浏览器通过 WebSocket 连接发送 JSON 消息：`{"type": "danmaku", "content": "弹幕内容", "color": "#FFFFFF", "position": "scroll"}`。

- 软件回答信息：

操作成功时，系统返回统一 JSON 响应：`{"code": 0, "message": "success", "data": {}}`。例如登录成功返回：`{"code": 0, "message": "登录成功", "data": {"token": "eyJhbGciOiJIUzI1NiIs... ", "user": {"userId": 1, "nickname": "用户昵称"}}`。

操作失败时，返回错误响应：`{"code": 非0错误码, "message": "错误描述", "data": null}`。例如账号密码错误返回：`{"code": 1001, "message": "账号或密码错误", "data": null}`；请求参数错误返回：`{"code": 400, "message": "参数校验失败：标题不能为空", "data": null}`；服务器内部错误返回：`{"code": 500, "message": "服务器内部错误，请稍后重试", "data": null}`。

分页查询成功时，返回：`{"code": 0, "message": "success", "data": {"list": [...], "page": 1, "pageSize": 20, "total": 100}}`。

加载状态反馈：前端请求发送后，按钮显示加载动画（如旋转图标），输入框禁用，防止重复提交。请求完成后恢复。

进度反馈：视频上传时，前端显示进度条（百分比和已上传大小/总大小），实时更新。转码处理时，显示“正在处理中”状态提示。

- 硬件控制面板定义：

本系统为纯软件系统，用户通过标准计算机外设（键盘、鼠标、显示器、扬声器）操作，不提供专门的硬件控制面板。视频播放时，用户可通过键盘快捷键控制：空格键播放/暂停，左右方向键快进/快退 5 秒，上下方向键调节音量，F 键全屏，Esc 键退出全屏。

3.2.2 与其他软件、硬件接口

- 与 MySQL 数据库的软件接口：

本系统后端通过 JDBC 驱动与 MySQL 数据库建立连接，进行数据的持久化存储。接口安排为：Spring Boot 应用启动时，HikariCP 连接池初始化，创建与数据库的长连接。业务处理时，通过 MyBatis 或 JPA 框架执行 SQL 操作。接口关系为：用户模块调用用户表，视频模块调用视频表，直播模块调用直播间表，弹幕模块调用弹幕表，评论模块调用评论表，关注模块调用关注表。各模块通过各自的数据访问对象（DAO/Mapper）与数据库交互，避免直接跨模块操作数据表。

连接配置参数：数据库服务器地址 `localhost:3306`，数据库名称 `ovls_db`，用户名和密码存储于 `application.yml`，连接池最大连接数 20，最小空闲连接 5，连接超时 30 秒。

- 与 Redis 缓存的软件接口：

本系统后端通过 Lettuce 或 Jedis 客户端与 Redis 服务建立连接，进行热点数据缓存。接口安排为：Spring Boot 应用启动时，Redis 连接工厂初始化。业务处理时，通过 `RedisTemplate` 或 `StringRedisTemplate` 执行缓存操作。接口关系为：用户模块缓存会话令牌和用户信息，视频模块缓存热度排行和播放计数，推荐模块缓存热门列表，弹幕模块缓存近期弹幕（可选）。

连接配置参数：Redis 服务器地址 `localhost:6379`，密码可选，数据库索引 0，连接超时 10 秒。

- 与 Nginx Web 服务器的软件接口：

本系统部署时，Nginx 作为反向代理和静态资源服务器。接口安排为：Nginx 监听 80 端口，接收所有客户端请求。静态资源请求（HTML、CSS、JS、图片、视频文件）由 Nginx 直接返回。API 请求（路径以 `/api/` 开头）转发至后端 Spring Boot 应用（`proxy_pass http://localhost:8080`）。WebSocket 连接（路径以 `/ws/` 开头）转发至 WebSocket 服务（`proxy_pass http://localhost:8081`，并配置 `proxy_set_header Upgrade $http_upgrade` 和 `proxy_set_header Connection "upgrade"` 支持协议升级）。

- 与 SRS 直播服务器的软件接口：

本系统通过 SRS 的 HTTP API 管理直播生命周期。接口安排为：创建直播间时，后端调用 SRS 的 `POST /api/v1/clients` 或流创建接口，获取推流地址（`rtmp://server/live/streamKey`）和拉流地址（`http://server/live/streamKey.flv`）。查询状态时，调用统计接口获取在线人数、码率信息。结束直播时，可选择调用断开接口或等待超时自动清理。接口关系为：直播模块与 SRS 交互，其他模块不直接调用。

SRS 部署于同一服务器或独立服务器，通过内网 IP 通信，HTTP API 端口 1985，RTMP 端口 1935，HTTP-FLV 端口 8080。

- 与 FFmpeg 视频处理工具的软件接口：

本系统后端通过 Java 的 `ProcessBuilder` 或 `Runtime.exec` 调用 FFmpeg 命令行工具，执行视频转码任务。接口安排为：视频上传完成后，后端拼接 FFmpeg 命令，如 `ffmpeg -i input.mp4 -vf "scale=1280:720" -c:v libx264 -preset fast -crf 23 output_720p.mp4`，启动子进程执行。执行完成后，读取输出文件，移至正式目录。接口关系为：视频模块调用 FFmpeg，其他模块不直接调用。

FFmpeg 安装于服务器，后端通过系统路径调用，需配置可执行文件位置。

- 与文件系统的硬件接口：

本系统通过 Java NIO 或 Apache Commons IO 库与服务器本地磁盘交互，存储视频文件和封面图片。接口安排为：视频上传时，后端接收字节流，写入临时目录（如 `/tmp/ovls/upload/`）。转码完成后，读取临时文件，写入正式目录（如 `/data/ovls/video/2026/05/video_12345/`）。视频播放时，后端根据视频 ID 拼接路径，读取文件返回流数据，或返回文件 URL 由 Nginx 直接提供。封面图片存储于 `/data/ovls/cover/` 目录。

存储设备为服务器本地硬盘或挂载的网络存储，文件系统类型 EXT4 或 XFS，单盘容量不低于 100GB，视频文件总大小预计 50GB 以内。

- 与客户端浏览器的硬件接口：

用户通过个人计算机或移动设备访问系统，设备需配备：显示器（分辨率不低于 1280x720，支持 HTML5 视频播放）、网络连接（带宽不低于 2Mbps，支持流畅视频播放）、输入设备（键盘、鼠标或触摸屏）、音频输出设备（扬声器或耳机，支持视频声音播放）。直播推流时，主播设备需额外配备：摄像头（分辨率不低于 720P）、麦克风（支持音频采集）。

3.3 系统数据结构设计

3.3.1 逻辑结构设计要点

本系统主要数据结构包括：

- 用户数据结构：包含用户ID、账号、密码、昵称、头像URL、简介、注册时间、用户类型（普通/创作者）。
- 视频数据结构：包含视频ID、标题、描述、标签、封面URL、视频URL（多清晰度）、时长、播放量、点赞数、评论数、上传者ID、上传时间、审核状态。
- 直播数据结构：包含直播间ID、标题、分类、封面、主播ID、开始时间、结束时间、在线人数、直播状态（准备中/直播中/已结束）。
- 弹幕数据结构：包含弹幕ID、内容、颜色、位置、发送者ID、直播间或视频ID、发送时间。
- 评论数据结构：包含评论ID、内容、发送者ID、目标视频ID、父评论ID（用于回复）、发送时间、点赞数。
- 关注数据结构：包含关注者ID、被关注者ID、关注时间。

各数据结构之间的关系：用户与视频为一对多（一个用户可上传多个视频）；用户与直播间为一对多（一个用户可开启多个直播，但同一时间只能有一个直播中）；视频与弹幕为一对多；视频与评论为一对多；用户与用户为多对多（关注关系）。

3.3.2 物理结构设计要点

用户表、视频表、直播表等核心表按ID建立主键索引，按创建时间建立普通索引加速查询。视频标签等搜索字段建立全文索引。用户账号建立唯一索引防止重复。

视频文件按目录结构存储：`/upload/video/年份/月份/视频ID/清晰度/文件名.mp4`。封面图片存储在：`/upload/cover/视频ID.jpg`。

3.3.3 数据结构与程序关系

用户模块操作用户表和关注表，注册时插入用户记录，登录时查询验证，关注时插入关注记录。

视频模块操作视频表和文件系统，上传时写入视频记录和文件，播放时查询记录读取文件，搜索时查询视频表。

直播模块操作直播表和直播服务器，开播时创建直播记录和直播间，观看时查询记录拉取流，结束时更新状态。

弹幕模块操作弹幕表和WebSocket连接，发送时插入记录并广播，显示时查询记录或接收实时推送。

推荐模块操作视频表和Redis缓存，计算热度时更新缓存，获取推荐时查询缓存或数据库。

4 系统数据库设计

用户表 (t_user)：存储用户基本信息。字段包括用户ID (主键，自增)、账号 (唯一，字符串)、密码 (加密存储，字符串)、昵称 (字符串)、头像URL (字符串)、简介 (文本)、用户类型 (枚举：普通/创作者)、注册时间 (日期时间)、状态 (枚举：正常/封禁)。

视频表 (t_video)：存储视频信息。字段包括视频ID (主键，自增)、标题 (字符串)、描述 (文本)、标签 (字符串，逗号分隔)、封面URL (字符串)、视频URL (字符串，JSON格式存储多清晰度)、时长 (整数，秒)、播放量 (整数)、点赞数 (整数)、评论数 (整数)、上传者ID (外键，关联用户表)、上传时间 (日期时间)、审核状态 (枚举：待审核/已通过/未通过)。

直播间表 (t_live_room)：存储直播信息。字段包括直播间ID (主键，自增)、标题 (字符串)、分类 (字符串)、封面 (字符串)、主播ID (外键，关联用户表)、推流地址 (字符串)、开始时间 (日期时间)、结束时间 (日期时间)、在线人数 (整数)、状态 (枚举：准备中/直播中/已结束)。

弹幕表 (t_danmaku)：存储弹幕记录。字段包括弹幕ID (主键，自增)、内容 (字符串)、颜色 (字符串)、位置 (枚举：滚动/顶部/底部)、发送者ID (外键)、目标ID (整数，视频ID或直播间ID)、目标类型 (枚举：视频/直播)、发送时间 (日期时间)。

评论表 (t_comment)：存储评论信息。字段包括评论ID (主键，自增)、内容 (文本)、发送者ID (外键)、视频ID (外键)、父评论ID (自关联，可为空)、发送时间 (日期时间)、点赞数 (整数)。

关注表 (t_follow)：存储关注关系。字段包括关注者ID (外键，关联用户表)、被关注者ID (外键，关联用户表)、关注时间 (日期时间)。复合主键为 (关注者ID，被关注者ID)。

5 运行设计

5.1 运行模块组合

系统运行时，各模块按用户操作组合调用：

- 浏览场景：

前端首页模块调用后端推荐模块获取视频列表，调用视频模块获取封面图片，渲染页面。

- 播放场景：

前端播放模块调用后端视频模块获取视频地址，调用弹幕模块获取历史弹幕，建立WebSocket接收实时弹幕，同时调用评论模块获取评论列表。

- 直播场景：

前端直播模块调用后端直播模块获取直播间信息和推流地址，建立WebSocket连接收发弹幕和指令，调用后端直播服务器获取直播流。

- 上传场景：

前端上传模块调用后端视频模块上传文件，后端调用转码模块处理视频，完成后更新数据库。

5.2运行控制

用户操作触发前端请求，前端按业务逻辑调用一个或多个后端接口。后端接口按权限验证、参数校验、业务处理、数据访问的顺序执行。关键操作如支付、删除需二次确认。并发请求通过数据库锁或乐观锁控制数据一致性。

5.3运行时间

用户注册登录响应时间不超过1秒。视频列表加载不超过2秒。视频首帧播放不超过3秒。弹幕发送显示延迟不超过1秒。直播推流到观众看到画面延迟不超过5秒。页面常规操作响应时间在毫秒级。